

AgentOpsBench: High-Throughput Agentic AI for Battery Analytics

Siddharth Gowda
Data Science Institute
Columbia University
New York, NY
scg2178@columbia.edu

Rushin Bhatt
Data Science Institute
Columbia University
New York, NY
rsb2213@columbia.edu

Aryaman Agrawal
Dept. of Electrical Engineering
Columbia University
New York, NY
aa5775@columbia.edu

Winston Li
Data Science Institute
Columbia University
New York, NY
wl3062@columbia.edu

Dhaval Patel
IBM TJ Watson Research
IBM
Yorktown Heights, NY
pateldha@us.ibm.com

Abstract—AssetOpsBench is an open framework for evaluating AI agents on industrial asset operations, currently scoped to HVAC systems. We extend it to lithium-ion battery systems by integrating NASA PCOE Li-ion cycling data into a CouchDB telemetry layer, constructing a 15-scenario benchmark across diverse stakeholder roles (fleet operators, BMS engineers, warranty analysts, thermal designers), and building a dedicated Battery MCP server with 11 tool functions for degradation analysis, remaining useful life (RUL) prediction, voltage milestone estimation, impedance tracking, fleet outlier detection, and LLM-narrated diagnosis. To address the performance costs of MCP stdio transport, we profile scenario execution and implement targeted optimizations: parallel CouchDB prefetch, MCP-level tool batching, and a disk-backed cache that persists RUL trajectories across subprocess boundaries. The disk cache reduces predict time from 7.0s to 0.002s, and MCP-level batching yields a $6\times$ speedup on fleet queries by consolidating 10 subprocess spawns into one. The framework required only minimal changes to the upstream plan-execute orchestration, adding support for multi-asset queries, demonstrating that the MCP multi-agent architecture extends to new physical asset domains.

Keywords: asset operations management, multi-agent systems, Model Context Protocol, battery prognostics, remaining useful life, CouchDB, TensorFlow, benchmark

I. INTRODUCTION

Industrial asset operations management (AOM) covers the continuous monitoring, fault detection, prognostics, and maintenance scheduling of physical assets at scale. AI-driven approaches are transforming this domain, but rigorous evaluation frameworks that reflect the full agentic reasoning pipeline, from data retrieval through multi-step planning to maintenance action generation, remain scarce. AssetOpsBench [1] addresses this by providing a multi-agent infrastructure built on the Model Context Protocol (MCP), a plan-execute orchestration layer, and evaluation scenarios grounded in industrial operations. While it has benchmark aspects, it is better understood as a theoretical framework and reference implementation

for how agents can be deployed in industrial and datacenter-related applications.

However, the upstream framework covers only HVAC assets such as chillers and air handling units. Lithium-ion battery systems are a distinct asset class with applications in grid-scale energy storage, electric vehicle fleets, uninterruptible power supplies, and aerospace [12]. Battery degradation involves coupled mechanisms: capacity fade, internal resistance growth, lithium plating, electrolyte decomposition, and thermal runaway, each with distinct electrochemical signatures [7]. Reasoning over these signals requires domain-specific tools that the upstream framework does not provide.

We extended AssetOpsBench to lithium-ion batteries by building a new Battery MCP server for RUL prediction, voltage milestone estimation, and fleet outlier detection. A second problem followed: given the architectural constraints of MCP stdio transport, how do we optimize this system for acceptable performance? We address both through a public fork (<https://github.com/siddharthgowda/AssetOpsBench>), contributing:

- A **CouchDB-backed telemetry layer** seeded from NASA PCOE Li-ion cycling data (34 cells, B0005–B0056), with cycle-type partitioning.
- A **Battery MCP server** with 11 tool functions for fleet prognostics, voltage and impedance analytics, and LLM-narrated diagnosis.
- **Pretrained prognostic heads** from Hsu et al. [7] for RUL and voltage prediction from early-cycle data, loaded via a Keras 2 compatibility layer.
- A **15-scenario benchmark** spanning diverse stakeholder roles and multi-step reasoning demands.
- A **profiling harness** decomposing agent latency into LLM, tool, and inference components, with targeted optimizations addressing the bottlenecks identified.

The MCP multi-agent architecture generalizes to new asset

domains with minimal effort. Adding batteries required a new MCP server, data seeding scripts, and small changes to support multi-asset queries. The upstream orchestration, LiteLLM backend, and existing servers remained unchanged.

The remainder of this paper is organized as follows. Section II surveys related work. Section III describes the upstream AssetOpsBench framework. Section IV presents our methodology, covering scenario design, framework integration, the prognostic model pipeline, and optimization strategy. Section V covers experimental setup. Section VI presents scenario results. Section VII analyzes profiling and optimization performance. Section VIII discusses limitations, and Section IX concludes.

II. RELATED WORK

A. Model Context Protocol and Tool-Augmented LLMs

The Model Context Protocol (MCP) provides a standardized interface for connecting LLMs to external tools and data sources via server-client communication [22]. FastMCP enables rapid server construction with typed tool definitions. In the AssetOpsBench framework, MCP servers encapsulate domain knowledge and computational tools, with a LiteLLM-backed planner generating structured sequences of tool calls using a ReAct-style loop [21]. This architecture separates reasoning (LLM) from computation (tools), enabling domain extension without modifying the orchestration layer, which is the key property we exploit in this work.

B. AssetOpsBench

AssetOpsBench [1], [3] is the most directly relevant prior work. It frames industrial AOM tasks as agent trajectories evaluated by an LLM judge across dimensions including task completion, data grounding, and hallucination risk. Its architecture consists of FastMCP domain servers (IoT, FMSR, TSFM, WO), a plan-execute orchestrator backed by a configurable LiteLLM endpoint, and a shared LiteLLM backend for both planning and tool-internal LLM calls. The TSFM server is relevant to our work: it wraps TinyTimeMixer-style time series foundation models for forecasting, anomaly detection, and RUL estimation, and these models are small enough to run on CPU, a design choice we mirror in our battery prognostic model. Related benchmarking work includes PHMForge [9] and broader agent benchmarking efforts [10], [11], [23].

C. Battery Prognostics and the RUL Prediction Model

Battery state-of-health (SOH) and remaining useful life (RUL) prediction from cycling data is a well-studied problem in PHM [20]. Classical approaches include empirical capacity fade models, electrochemical impedance spectroscopy, and Kalman filter-based state estimation [19]. Data-driven approaches have achieved strong results on standardized datasets including the NASA PCOE Li-ion aging dataset [5], the CALCE dataset, and the recently released BatteryLife benchmark [8]. Early-cycle features are predictive of long-term cycle life [14], [15], [17], and sequence models have also been

applied [16]. Damage propagation modeling has been studied in related prognostics contexts [18].

The model at the core of our Battery MCP server is the deep neural network from Hsu et al. [7], implemented in the open-source acctouhou repository [6]. This architecture uses charge and discharge profiles from as few as one early cycle to accurately predict battery life and voltage behavior across the full aging trajectory. We chose this model because it is lightweight and runs on CPU, consistent with the design philosophy of the upstream TSFM server. Our contribution is in systems integration: wrapping these frozen pretrained weights with a TF 2.16+ compatibility layer, batched inference paths, CouchDB I/O parallelism, and MCP tool abstractions. We do not propose a new neural architecture.

D. Why Batteries Are an Important Use Case

Lithium-ion batteries underpin infrastructure across sectors: electric vehicles (range, warranty, resale), grid-scale storage (efficiency, safety), and data center UPS (uptime). Yet most battery analytics tools remain standalone scripts or proprietary systems, disconnected from agentic frameworks that can reason across data streams and generate maintenance actions. The same plan-execute pattern that dispatches a work order for a faulty HVAC chiller can diagnose a degrading battery cell and recommend a replacement schedule. Battery cycling data is well-suited to multi-step agent reasoning: capacity fade is predictable from early cycles [14], impedance growth tracks electrolyte aging, voltage profiles shift with lithium plating, and temperature correlates with accelerated degradation.

III. BACKGROUND: ASSETOPSBENCH

AssetOpsBench [1] is an open framework for deploying AI agents on industrial asset operations tasks. Its architecture has four components.

MCP Domain Servers. FastMCP servers expose typed tool functions for specific domains. The IoT server retrieves asset and sensor data from CouchDB. The FMSR server maps signals to failure mode taxonomies. The TSFM server performs forecasting, anomaly detection, and data quality assessment using TinyTimeMixer-style models. The WO server generates maintenance actions.

Plan-Execute Orchestration. A sequential planner-executor pattern where the planner generates a structured multi-step plan as a sequence of (server, tool, args) triples. The executor dispatches each step via MCP, aggregates results, and passes context forward.

LiteLLM Backend. A configurable LLM interface supporting multiple providers via environment variables, shared across both planning and tool-internal LLM calls such as diagnosis narration.

Evaluation Infrastructure. Scenarios are expressed as natural-language queries; agent trajectories are evaluated using an LLM-as-judge framework examining task completion, data retrieval correctness, logical reasoning quality, and hallucination risk [23].

IV. METHODOLOGY

A. Scenario Design

Our 15 battery scenarios differ from the upstream AssetOpsBench benchmarks in structure and complexity. Upstream scenarios tend to be concise, asset-specific queries. The user names a particular chiller or air handling unit and asks a focused technical question. Our scenarios are conversational, context-rich prompts written from the perspective of named stakeholder roles (fleet operators, BMS engineers, warranty analysts, quality assurance leads). Rather than referencing a single asset by identifier, these queries describe an operational problem in domain-specific language, reference concepts such as capacity fade thresholds or impedance growth trends, and require the agent to plan a multi-step analysis across several tools.

This design reflects how domain experts in practice interact with analytical systems. A supply chain director, for example, does not issue a structured API call. They describe a business problem: *“We run a massive logistics operation, and unexpected asset downtime destroys our margins. Rank the entire fleet by remaining useful life and highlight the units that need replacement in the next 30 days.”* The agent must parse this intent, identify the appropriate tools, and chain them into a coherent execution plan without explicit instruction about which tools to call or in what order.

As an illustrative example, in the Manufacturing QA scenario a quality assurance lead asks the system to flag cells degrading faster than expected within the first 50 cycles. The agent retrieves cycle summaries across the fleet, runs statistical outlier detection on early-cycle fade rates, and completes the interaction in under 18 seconds on CPU. Table I summarizes all 15 scenario categories.

B. Integrating Battery Support into AssetOpsBench

1) *From Shared Servers to a Dedicated Battery Server:* Our initial approach was to integrate battery tools directly into the existing MCP servers, following the principle of minimizing changes to the upstream codebase. We considered adding RUL estimation as a new function within the time series server and battery metadata retrieval within the IoT server. In practice, however, battery analytics proved to be highly domain-specific. The data formats, preprocessing requirements, model interfaces, and failure mode taxonomies differ from the HVAC-oriented tools already in place. Embedding battery logic within shared servers introduced tight coupling and made it difficult to iterate on battery functionality without risking regressions on existing capabilities.

We therefore built a standalone Battery MCP server exposing 11 composable tool functions for fleet listing, RUL prediction, voltage curve and milestone estimation, impedance trajectory analysis, capacity outlier detection, and LLM-narrated diagnosis. This server follows the same FastMCP stdio transport pattern as all other AssetOpsBench domain servers. The modular approach is more scalable. New battery tools can be added without modifying any existing server, and

TABLE I
BATTERY BENCHMARK SCENARIO CATEGORIES (15 TOTAL).

Stakeholder	Reasoning Demand
Fleet Operator	Fleet-wide RUL ranking, outlier flagging, maintenance prioritization.
BMS Engineer	Cell-level degradation curves, impedance trajectories, lithium plating indicators.
Warranty Analyst	Capacity fade timelines vs. warranty thresholds; financial exposure estimation.
Thermal / Power Electronics	Temperature-correlated degradation sensitivity; cycle-phase thermal profiles.
Edge Deployment	Inference latency characterization for constrained deployment scenarios.
Microgrid / Second-Life	SOH thresholds for repurposing; remaining capacity estimation post-EOL.
Safety / EIS	Thermal runaway risk indicators; impedance growth alarm triggers.
Manufacturing QA	Early-cycle outlier detection for incoming cell acceptance testing [14].
Environmental / Chambers	Correlation of ambient condition variables with degradation rate.
Logistics / Procurement	Fleet-level health summary for asset inventory management.
Auxiliary LV Cutoffs	Voltage milestone crossings vs. system cut-off thresholds.
Lab / Sensor Drift	Data quality and measurement consistency checks.
Consultant Benchmarking	Comparative analysis across cell chemistries and aging conditions [15].
Used-Asset Valuation	SOH-adjusted residual value estimation for asset re-sale [17].
Multi-domain (TSFM + Battery)	Combined foundation-model forecast with DNN-RUL cross-validation.

the same pattern extends to future asset classes. The full tool set is listed in Appendix Table V.

To enable the agent to reason about battery-specific fault conditions, we also extended the upstream failure mode taxonomy with five battery degradation modes: capacity fade, internal resistance increase, thermal runaway, lithium plating, and electrolyte decomposition [20]. Each mode is associated with observable symptoms and diagnostic indicators, allowing the planner to route battery fault queries through the failure mode server for classification before or after invoking battery-specific tools.

2) *Supporting Multi-Asset Queries:* The upstream AssetOpsBench implementation was designed around single-asset queries, where each tool call operates on one device at a time. Battery scenarios frequently require fleet-level reasoning, such as ranking all cells by remaining useful life or detecting outliers across a batch. Supporting this required several targeted changes. We emphasized the foreach pattern and bulk MCP calls in the system prompt so the planner knows how to handle multi-asset requests. We designed tool interfaces to accept lists of asset identifiers natively, so that a single call to the batched RUL predictor handles the entire fleet. For tools that operate on a single asset, we added a loop mechanism in the executor so the planner can express “run this for each cell” without enumerating every call. These modifications were minimal and did not change the core plan-execute orchestration logic.

C. Prognostic Model and Data Pipeline

Remaining useful life prediction for lithium-ion batteries requires modeling complex electrochemical degradation patterns that are not amenable to simple statistical functions. At the same time, we did not need a large neural network for this task. The inference needs to be fast and CPU-friendly, consistent with the design philosophy of the existing time series server in AssetOpsBench [7]. We selected the pretrained deep neural network from Hsu et al. [7], available in the acctouhou repository [6], which predicts both remaining useful life and voltage behavior from early charge-discharge cycles. It runs on CPU with sub-second inference per cell. We use the pretrained weights without modification.

Our battery data comes from the NASA PCOE Li-ion Battery Aging Dataset [5], which provides cycling records for 34 cells across discharge, charge, and impedance measurement types. The cells are 18650-format NCA chemistry, cycled under varying conditions until end of life. Of these, 10 have sufficiently complete paired profiles (at least 100 cycles each) to serve as model inputs. The raw NASA JSON files are loaded into CouchDB with documents partitioned by cycle type, and we generate a CSV bridge that allows the TSFM server to operate on battery data without modification.

The inference pipeline processes raw cycling data through several stages. For each cycle, four measurement channels (cumulative capacity, voltage, current, and temperature) are extracted and interpolated to a fixed-length representation. The first 100 paired charge-discharge cycles for a given cell are assembled into arrays and augmented with per-cycle summary statistics. These inputs are normalized using pretrained statistics stored alongside the model weights. The normalized tensors pass through dedicated feature extraction networks for charge and discharge data, and the resulting features are concatenated with the normalized summary statistics. The combined representation is arranged into sliding windows that capture the cell’s recent history and fed to the RUL prediction head, which outputs an estimated remaining cycle count at each point in the cell’s aging trajectory. A separate voltage prediction head follows the same pipeline with an additional state-of-charge input. The pretrained weights were serialized under Keras 2. We load them via the `tf_keras` compatibility package, registering custom activations as needed.

The Hsu et al. model was trained on Severson LFP chemistry data [14], while the NASA cells are NCA. This domain shift means absolute RUL values carry error without fine-tuning, though relative rankings across cells remain informative for fleet-level decision-making.

D. Optimization Strategy

We derived our optimization targets from profiling rather than theory. We instrumented scenario executions end-to-end, decomposing total latency into planning, tool execution, and summary generation, and used these profiles to identify where time went. The profiler was lightweight, using wall-clock timing rather than GPU profiling or tracing. Our system is CPU-bound, the LLM runs on a remote endpoint, and the

costs are I/O and process-level overhead, not kernel-level computation.

The optimization surface is narrow. The prognostic model is small with modest input sizes, leaving little room to accelerate inference. The LLM planner is not locally tunable. The viable targets are CouchDB network I/O (required on every call) and redundant recomputation imposed by the MCP subprocess model, which destroys all in-process state between tool invocations. We targeted both with parallel CouchDB prefetch to overlap data retrieval, a disk-backed cache to persist trajectories across subprocess lifetimes, and batched tool interfaces to reduce subprocess spawns for fleet queries. We also tested graph precompilation and batched TensorFlow inference. Profiling results and ablation methodology are in Section VII.

V. EXPERIMENTAL SETUP

Hardware. All experiments were run on an Apple M4 Max (ARM, 14 logical cores, 36 GiB unified memory). TensorFlow ran CPU-only with no Metal or CUDA backend installed. Timing results should be interpreted relative to this platform. Numbers will differ on x86 or GPU-equipped hardware.

Software. Python 3.12.10, TensorFlow 2.21.0 [24] with the `tf_keras` compatibility package for Keras 2 weight loading, FastMCP, LiteLLM, and CouchDB (Docker).

LLM Backend. All planning and tool-internal LLM calls use Llama 3.1 (8B) hosted via Cerebras, accessed through the shared LiteLLM backend. We migrated to this endpoint after encountering access issues with our original IBM Watsonx API key. LLM latency is treated as a fixed cost since the model runs on a remote endpoint and is not locally tunable.

Data. NASA PCOE Li-ion dataset [5], 10 cells (B0005, B0006, B0007, B0018, B0033, B0034, B0036, B0054, B0055, B0056) seeded to CouchDB, each with 100 charge and 100 discharge documents. These 10 cells were selected because they have sufficiently complete paired charge-discharge profiles to meet the prognostic model’s minimum input requirement of 100 cycles; the remaining cells in the NASA dataset lack enough usable data to produce meaningful predictions.

Pretrained Weights. RUL and voltage prediction heads with corresponding normalization arrays, obtained from the acctouhou repository [6] and loaded at server startup.

Profiling Approach. Because the prognostic model is small and runs on CPU, and the dominant latency costs are I/O and subprocess overhead, we use wall-clock timing rather than GPU profiling tools.

VI. RESULTS

A. Scenario Evaluation

We evaluated all 15 benchmark scenarios end-to-end using the plan-execute pipeline with Llama 3.1 (8B) hosted on Cerebras. Four scenarios (3, 11, 14, 15) were excluded because they exceeded the Cerebras endpoint’s 32,768-token context window during the synthesis step. This is a provider-side constraint unrelated to our framework.

Of the remaining 11 scenarios, Table II summarizes the outcomes. A scenario is considered passing if the agent selects the relevant tools, retrieves grounded data, and produces a response based on tool output.

TABLE II
SCENARIO EVALUATION RESULTS (EXCLUDING 4 CONTEXT-OVERFLOW SCENARIOS).

Persona	Result	Notes
BMS Hardware	Partial	Phantom tool calls for MAE/RMSE synthesis
Warranty	Pass	Capacity fade timeline grounded in data
Edge AI	Pass	TSFM tools executed successfully
Autonomous Fleet	Partial	Sparse plan with only one prediction tool for a fleet query
Supply Chain	Pass	Fleet RUL ranking completed correctly
Aux Controller	Pass	Voltage milestone comparison returned
QA Lead	Pass	Outlier detection completed
Env Chamber	Pass	Temperature-correlated analysis completed
Microgrid	Pass	SOH threshold analysis grounded in data
Used Asset	Pass	SOH-adjusted valuation grounded in data
Lab Engineer	Partial	Planner did not invoke TSFM server for impedance modeling

Of the 11 evaluated scenarios, 8 passed and 3 produced partial results. The tool infrastructure executed correctly across all scenarios. The battery MCP server and statistical tools returned valid, grounded output without errors, and cells outside the model-ready set returned null cleanly. The partial cases were all planning-level issues. The planner either generated phantom tool calls (referencing tools not in the registry), produced sparse plans that underutilized the available tools, or failed to coordinate across multiple servers for multi-domain queries.

These outcomes were anticipated. The upstream AssetOps-Bench framework has known limitations, and through discussions with our IBM mentor we understood that perfect scenario completion was not expected. Our scenarios are more complex than the upstream benchmarks, which reference specific assets by identifier and request single operations. Our scenarios are open-ended, domain-rich prompts requiring the planner to infer multi-step tool sequences without guidance. That 8 of 11 passed and the remaining partial results were all attributable to planning rather than tool infrastructure confirms that the server design and tool implementations are sound.

B. RUL Prediction Accuracy

Quantitative RUL MAE evaluation requires complete charge/discharge profiles and documented end-of-life cycles from the NASA dataset. Ground truth curation for the 10 deployed cells is left as future work.

VII. HPML ANALYSIS

A. Scenario Profiling

We instrumented the plan-execute pipeline to record wall-clock timing for each component, including discovery, planning, per-tool execution, and summary generation. Tool calls dominated the latency budget, with RUL prediction averaging roughly 10 seconds per invocation compared to 3–4 seconds for statistical tools. Planning and summary generation together accounted for under 2 seconds. The ablation study below isolates the fetch and predict components within each RUL call.

B. MCP-Level Batching

Before introducing the batched RUL predictor, fleet-level scenarios required the planner to issue a separate tool call for each cell. Table III compares the two approaches for a 10-cell fleet query.

TABLE III
MCP-LEVEL BATCHING: PER-CELL TOOL CALLS VS. A SINGLE FLEET-BATCHED CALL, $N=10$ CELLS.

Strategy	Wall (s)	Subprocess Spawns	Speedup
Per-cell (10 separate tool calls)	44.64	10	baseline
Fleet-batched (1 tool call)	7.37	1	$6.06\times$

The $6\times$ speedup is proportional to the number of eliminated subprocess spawns, confirming that per-call overhead dominates inference cost. This differs from the *batched predict* optimization in the ablation below. MCP-level batching reduces subprocess spawns by consolidating a fleet query into one tool call, while batched predict operates *within* a single call by concatenating input tensors for fewer TensorFlow predict calls. MCP-level batching yields real gains. Batched predict has no effect at this model size.

C. Optimization Ablation

To isolate the contribution of each optimization, we ran the RUL prediction pipeline for all 10 cells under five configurations, enabling one additional optimization at each step. Each configuration was repeated 3 times and timing was decomposed into fetch (CouchDB retrieval) and predict (TensorFlow inference) components. Table IV reports the results.

TABLE IV
OPTIMIZATION ABLATION, $N=10$ CELLS, 3 REPEATS, APPLE M4 MAX CPU. EACH ROW ADDS ONE OPTIMIZATION TO THE PREVIOUS CONFIGURATION.

Configuration	Wall (s)	Fetch (s)	Predict (s)
Naive baseline	12.64	5.655	6.985
+ Parallel fetch	11.18	3.852	7.325
+ Graph precompile	10.88	3.859	7.025
+ Batched predict	10.93	3.833	7.099
+ Disk cache	3.83	3.831	0.002

Parallel CouchDB fetch reduces data retrieval time by 32% (5.7s to 3.9s) by overlapping network requests via a thread pool. Graph precompilation and batched predict produced negligible improvements. Predict time fluctuated in the 7.0–7.3 second range regardless of configuration, consistent with the model being too small for batch-level kernel gains on CPU.

The disk cache is the most impactful optimization. After a first-pass prediction, each cell’s RUL trajectory is written to disk. On subsequent calls, if CouchDB data has not changed, the cached results are loaded directly and inference is skipped. This drops predict time from roughly 7 seconds to 0.002

seconds. The residual 3.8 seconds is CouchDB fetch time, which the cache does not eliminate.

VIII. DISCUSSION

Extending AssetOpsBench to batteries worked with a small planner model. A well-structured system prompt with foreach and bulk call patterns, combined with composable tool primitives, enabled Llama 3.1 (8B) to produce correct plans for 8 of 11 evaluated scenarios, with 3 partial results. All partial cases were planning-level issues, not tool infrastructure problems. Tool interface design and prompt quality matter as much as model scale. Small LLMs can generate workable plans for multi-asset queries when the tools are well-defined and the prompt provides sufficient guidance.

When using a small LLM as the planner, the dominant bottleneck is subprocess and network overhead, not model inference. Each MCP tool call spawns a fresh process, re-imports TensorFlow, reloads weights, and fetches data from CouchDB. Consolidating queries into fewer requests reduces both latency and network load and is the most impactful optimization in this architecture. For practitioners building similar MCP-based agent systems, minimizing the number of tool calls matters more than optimizing what happens inside each one.

Graph precompilation and batched predict produced negligible improvements because the prognostic model is a lightweight DNN. The TensorFlow graph is small enough that retracing adds minimal overhead, and per-cell inference is fast enough that batching does not help over a simple loop. These optimizations would matter for larger models or higher cell counts but are not bottlenecks at this scale.

The disk cache eliminates inference when it hits, reducing pipeline time by roughly half. We cannot fully characterize its practical impact because our scenario set is small (we built this benchmark from scratch). In operational settings, practitioners revisit the same assets over short time horizons. An engineer concerned about a fleet of cells this week will likely query those cells again in subsequent sessions. Cache reuse would be frequent under that pattern. Caching is straightforward here because the computation is deterministic. The same cycling data produces the same RUL trajectory, so there is no ambiguity about validity.

The prognostic model was trained on Severson LFP chemistry [14], while the NASA PCOE cells are NCA 18650. Both are lithium-based chemistries, so the learned degradation patterns transfer partially, but the differing electrode materials and aging characteristics mean absolute RUL predictions carry error without fine-tuning. The statistical tools (impedance growth, outlier detection) are unaffected. Fine-tuning on NCA data would improve accuracy and is a natural next step, but is beyond our current scope. We aimed to show that a pretrained model can be integrated into the MCP server architecture and invoked by the planner. Prediction quality depends on the model and its training data, not the integration.

This work covers only lithium-ion cells. Battery chemistries vary widely (LFP, solid-state, sodium-ion, lead-acid), each

with distinct degradation mechanisms and failure modes. Our tools and model are lithium-ion-specific and may not transfer to other chemistries without substantial changes. Extending to broader battery types would require chemistry-specific models and modified tool interfaces.

Even with the disk cache, each MCP tool call pays a residual 5–6 second cost for subprocess spawn, TensorFlow import, and weight loading. The MCP stdio transport model cannot eliminate this because each call is a fresh process. A persistent server architecture would remove this overhead and is a natural next step.

IX. CONCLUSION

We extended AssetOpsBench to lithium-ion batteries with a new Battery MCP server (11 tool functions), a CouchDB telemetry layer from NASA PCOE data, pretrained Hsu et al. [7] prognostics, a TSFM CSV bridge, and 15 stakeholder-driven benchmark scenarios.

Adding batteries required a new MCP server, data seeding, and small changes to support multi-asset queries. The upstream orchestration, LiteLLM backend, and existing servers remained unchanged.

Subprocess overhead per tool call drives latency, not TensorFlow inference. The disk cache eliminates redundant inference (predict time drops from 7s to 0.002s). MCP-level batching consolidates fleet queries into one subprocess spawn ($6\times$ speedup at $N=10$). Graph precompilation and batched predict had no measurable effect at this model size. The residual 5-second per-call cost from TensorFlow import and weight loading motivates persistent MCP server modes as future work.

Availability

The implementation is available as a public fork at: <https://github.com/siddharthgowda/AssetOpsBench> forked from the upstream IBM AssetOpsBench repository at <https://github.com/IBM/AssetOpsBench>.

REFERENCES

- [1] IBM Research, “AssetOpsBench,” 2025. [Online]. Available: <https://github.com/IBM/AssetOpsBench>
- [2] S. Gowda, R. Bhatt, A. Agrawal, W. Li, and D. Patel, “AssetOpsBench (Battery Extension Fork),” 2025. [Online]. Available: <https://github.com/siddharthgowda/AssetOpsBench>
- [3] IBM Research, “AssetOpsBench: A Benchmark for Agentic AI in Industrial Asset Operations,” arXiv:2506.03828, 2025. [Online]. Available: <https://arxiv.org/pdf/2506.03828>
- [4] IBM Research, “AssetOpsBench Dataset,” Hugging Face, 2025. [Online]. Available: <https://huggingface.co/datasets/ibm-research/AssetOpsBench>
- [5] NASA Prognostics Center of Excellence, “Li-Ion Battery Aging Datasets,” NASA PCOE Data Repository. [Online]. Available: <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>
- [6] acctouhou, “Prediction_of_battery,” GitHub, 2021. [Online]. Available: https://github.com/acctouhou/Prediction_of_battery
- [7] C.-W. Hsu, R. Xiong, N.-Y. Chen, J. Li, and N.-T. Tsou, “Deep neural network battery life and voltage prediction by using data of one cycle only,” *Applied Energy*, vol. 306, p. 118134, 2022. DOI: <https://doi.org/10.1016/j.apenergy.2021.118134>
- [8] R. Tan et al., “BatteryLife: A Comprehensive Dataset and Benchmark for Battery Life Prediction,” KDD, 2025. [Online]. Available: <https://github.com/Ruifeng-Tan/BatteryLife>

- [9] “PHMForge: A Benchmark for Prognostics and Health Management,” arXiv:2604.01532, 2026. [Online]. Available: <https://arxiv.org/pdf/2604.01532>
- [10] “Agent Benchmarking for Industrial Tasks,” OpenReview, 2025. [Online]. Available: <https://openreview.net/forum?id=fe8mzHwMxN>
- [11] “Multi-Agent Reasoning Systems,” OpenReview, 2025. [Online]. Available: <https://openreview.net/pdf?id=z53s5p0qhf>
- [12] IBM Research, “Revolutionizing Industries with AI-Powered Digital Twins,” 2024. [Online]. Available: <https://research.ibm.com/blog/revolutionizing-industries-with-ai-powered-digital-twins>
- [13] “AssetOpsBench Competition,” Codabench, 2025. [Online]. Available: <https://www.codabench.org/competitions/10206>
- [14] K. A. Severson et al., “Data-driven prediction of battery cycle life before capacity degradation,” *Nature Energy*, vol. 4, pp. 383–391, 2019.
- [15] P. M. Attia et al., “Statistical learning for accurate and interpretable battery lifetime prediction,” *arXiv preprint arXiv:2101.01885*, 2021.
- [16] S. Paradis and M. Whitmeyer, “Pay Attention: Leveraging sequence models to predict battery useful life,” *arXiv preprint arXiv:1910.01347*, 2019.
- [17] S. Stock et al., “Early quality classification and prediction of battery cycle life using machine learning,” *Energy AI*, 2022.
- [18] A. Saxena et al., “Damage propagation modeling for aircraft engine run-to-failure simulation,” *International Conference on Prognostics and Health Management*, 2008.
- [19] W. He et al., “State of charge estimation of lithium-ion battery using extended Kalman filter,” *Energy*, vol. 36, no. 6, pp. 3423–3430, 2011.
- [20] M. Bercibar et al., “Critical review of state of health estimation methods of Li-ion batteries for real applications,” *Renewable and Sustainable Energy Reviews*, vol. 56, pp. 572–587, 2016.
- [21] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2023.
- [22] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [23] Z. Liang et al., “AgentBench: Evaluating LLMs as agents,” *arXiv preprint arXiv:2308.03688*, 2023.
- [24] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2016.
- [25] D. Crankshaw et al., “Clipper: A low-latency online prediction serving system,” *NSDI*, 2017.
- [26] B. Lim et al., “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, 2021.

APPENDIX

Per the HPML AI Use Policy (posted on CourseWorks).

Did your team use any AI tool in completing this project?
Yes.

Tools used: Claude, Cursor.

Specific purpose: We used Claude to edit and compress prose in the report (abstract, methodology, discussion) and to generate the LaTeX system architecture diagram. We used Cursor to generate preprocessing code (`src/servers/battery/preprocessing.py`) using the original model repository (https://github.com/accouhou/Prediction_of_battery/tree/main) as context, and to write docstrings for the Battery MCP server and profiling modules (`src/servers/battery/main.py`, `benchmark_optimizations.py`, `mcp_batch_demo.py`, `profile_scenario.py`). We also used to cursor to help condense our writing in the readme and help create the repository structure in the readme since AssetOpsBench is quite large.

Sections affected: Report abstract, methodology, discussion, appendix architecture diagram, README. Code files listed above.

How we verified correctness: We re-read all edited prose and rewrote anything that did not match our intent. We reviewed all generated code against our own understanding of the system.

By submitting this project, the team confirms that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above.

Figure 1 illustrates the high-level system architecture. The AI orchestrator receives a user query, generates a multi-step plan, and executes it by dispatching MCP tool requests to the appropriate domain servers. The Battery MCP server (highlighted) is our addition to the existing services layer.

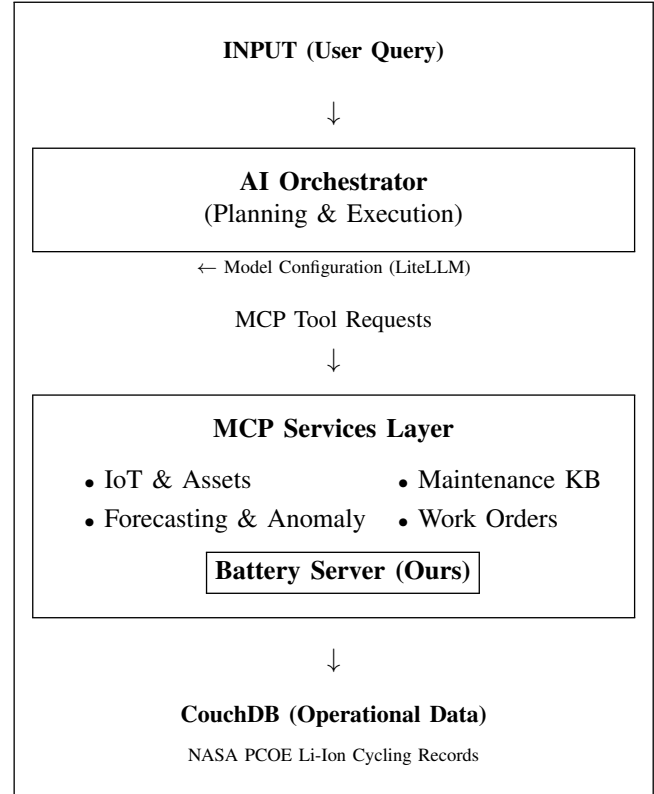


Fig. 1. AssetOpsBench system architecture (high level). The Battery MCP Server (boxed) is the fork-specific addition. All other components are upstream.

TABLE V
BATTERY MCP SERVER TOOL FUNCTIONS. CATEGORY: M = MODEL-BASED (PRETRAINED DNN), S = STATISTICAL/ANALYTIC, H = HYBRID (LLM + NUMERICAL).

#	Tool	Description	Cat.
1	list_batteries	Lists available battery assets with optional site-level filtering.	S
2	get_battery_cycle_summary	Returns per-cycle capacity, temperature, voltage, and impedance statistics from CouchDB.	S
3	predict_rul	Predicts cycles to 30% capacity fade (≈ 1.4 Ah EOL threshold) using the pretrained RUL head; reports inference time and computes MAE against ground truth when available.	M
4	predict_rul_batch	Fleet-batched RUL prediction over multiple cells in a single call, reducing tool-call round trips for fleet queries.	M
5	predict_voltage_curve	Returns a 500-point predicted discharge voltage curve at a given cycle index using the pretrained voltage head.	M
6	predict_voltage_milestones	Identifies the first predicted cycle crossing voltage thresholds (defaults: 2.9 V, 2.8 V, 2.7 V).	M
7	get_actual_voltage_milestones	Measured counterpart to tool 6; enables predicted-vs.-actual comparison on voltage threshold crossings.	S
8	get_impedance_trajectory	Returns per-cycle charge transfer resistance, electrolyte resistance, and rectified impedance magnitude.	S
9	analyze_impedance_growth	Fits an exponential trend to impedance data; triggers an alarm if the final-to-initial ratio exceeds a configurable threshold.	S
10	detect_capacity_outliers	Computes fleet-wide z-scores on early-window fade rates; flags cells with $ z > 2$ as anomalous.	S
11	diagnose_battery	Combines RUL, impedance, and capacity outputs with LLM-based narrative classification into degradation modes (capacity fade, lithium plating, impedance growth, or healthy) with severity and recommendations.	H